CARRERA: Licenciatura en Sistemas	**PLAN DE ESTUDIO:** 2025 (Res. "C.S." 274/24)
ASIGNATURA: Programación IV	**AÑO:** 3 er. año – 1.er cuatrimestre
CARGA HORARIA SEMANAL: 8 hs (128 hs totales)	**RÉGIMEN:** Teórico/Práctico
PROFESOR/A TITULAR / A CARGO: Ignacio Luis Novello Pelayo	**PROFESOR/A (S) AYUDANTE (S):**

1.-FUNDAMENTACIÓN

La transformación digital y la creciente demanda de soluciones tecnológicas escalables, seguras y orientadas al usuario han consolidado a los sistemas web como uno de los pilares fundamentales de los procesos organizacionales, gubernamentales y productivos. En este contexto, las aplicaciones distribuidas y paralelas constituyen la base técnica de plataformas modernas tales como servicios en la nube, sistemas transaccionales, aplicaciones móviles, motores de recomendación, tableros en tiempo real y herramientas colaborativas. La formación de profesionales capaces de comprender, diseñar e implementar estas soluciones resulta estratégica para el desarrollo regional y para la actualización constante del sistema educativo.

La presente asignatura se orienta a brindar a los estudiantes los conocimientos, habilidades y actitudes necesarias para desenvolverse de manera competente en entornos de desarrollo web actuales, caracterizados por arquitecturas cliente-servidor, comunicaciones asincrónicas, servicios distribuidos y módulos de procesamiento paralelo. Se busca promover la comprensión profunda de los conceptos fundamentales que sustentan estas tecnologías, a la vez que se desarrollan capacidades prácticas que permitan a los futuros profesionales construir aplicaciones robustas, eficientes y centradas en las necesidades de los usuarios.

La incorporación de contenidos tales como algoritmos distribuidos, programación paralela, diseño de interfaces y comunicación entre procesos responde a una necesidad concreta del sector productivo y del ámbito público, donde las aplicaciones web constituyen herramientas fundamentales para la gestión, la optimización de recursos y la toma de decisiones basada en datos. De este modo, la asignatura también fortalece la vinculación con el entorno, aportando una formación pertinente, actualizada y con impacto directo en el desarrollo local y regional.

En suma, esta asignatura contribuye al perfil profesional del plan de estudios al ofrecer una formación integral que articula fundamentos teóricos, capacidades técnicas y criterios de diseño centrados en la experiencia del usuario. Su enfoque interdisciplinario y orientado a competencias permite preparar estudiantes capaces de abordar proyectos web modernos con solvencia técnica y visión crítica fortaleciendo así la misión institucional de formar profesionales altamente calificados para la sociedad del conocimiento.

El eje articulador de la propuesta consiste en concebir a la Web como el sistema distribuido por excelencia. Desde la primera clase, cliente y servidor se presentan como nodos que intercambian mensajes a través de una red; el modelo asincrónico y orientado a eventos de JavaScript introduce de manera natural la problemática de la concurrencia; y la necesidad de que una aplicación web atienda a muchos usuarios en forma simultánea, confiable y escalable conduce a la programación paralela y a los algoritmos y arquitecturas distribuidas. De este modo, los contenidos mínimos no se abordan como bloques aislados, sino como etapas sucesivas y de complejidad creciente sobre un mismo objeto de estudio: una aplicación web que los estudiantes construyen y hacen evolucionar a lo largo del cuatrimestre.

Se adopta JavaScript, tanto en el navegador como en el entorno Node.js, como lenguaje principal de la asignatura. Esta decisión permite utilizar un único lenguaje en todas las capas de la

aplicación y concentrar el esfuerzo en los conceptos. Para los contenidos de concurrencia y paralelismo se incorporan ejemplos en otros lenguajes (por ejemplo, Python o Java), con el fin de contrastar distintos modelos de concurrencia y evaluar su adecuación a cada contexto, en línea con la competencia CE.AyL.4.

Programación IV aporta a las siguientes actividades reservadas del título Licenciado/a en Sistemas

- AR1. Especificar, proyectar y desarrollar sistemas de información, **sistemas de comunicación de datos** y software cuya utilización pueda afectar la seguridad, salud, bienes o derechos.
- AR2. Proyectar y dirigir lo referido a seguridad informática.
- AR3. Establecer métricas y normas de calidad de software.
- AR4. Certificar el funcionamiento, condición de uso o estado de lo mencionado anteriormente.
- AR5. Dirigir y controlar la implementación, operación y mantenimiento de lo anteriormente mencionado.

En base a ese aporte, la asignatura contribuye de manera directa a los siguientes alcances del título, según lo especificado en la Res. “C.S.” 274/24 que aprueba la carrera: AL1 (diseño, implementación, prueba y verificación de sistemas de software), AL3 (análisis y diseño de los sistemas de software que dan soporte a los sistemas de información de una organización), AL5 (métricas y normas de calidad y seguridad de software, aplicadas a la verificación y a la medición del rendimiento de las aplicaciones), AL6 (especificación, diseño y desarrollo de los componentes de seguridad del software de aplicación), AL9 (análisis y evaluación de la implementación y puesta a punto de sistemas de procesamiento de datos) y AL10 (sistemas de procesamiento de datos vinculados mediante redes de comunicaciones).

Asimismo, por integrar el trayecto del título intermedio de Programador/a de Sistemas, aporta a los alcances de dicho título vinculados con la definición de estrategias, técnicas y metodologías de programación; el desarrollo de sistemas que aseguren su funcionamiento según definiciones de performance establecidas; y el diseño e implementación de pruebas de funcionamiento, verificación y validación de software.

Articulación con otras asignaturas del plan de estudios

Articulación vertical. La asignatura recupera los saberes de Programación I, II y III (resolución algorítmica, estructuras de datos y análisis de eficiencia) y de Sistemas Operativos, donde los estudiantes abordaron procesos, concurrencia y sincronización desde la perspectiva del sistema operativo. Programación IV retoma esos conceptos desde la perspectiva del programador, centrándose en los modelos de concurrencia que ofrecen los lenguajes y plataformas actuales. Aplica, además, los contenidos de Interacción Humano-Computadora de Ingeniería de Software I al diseño concreto de interfaces web. A su vez, sienta las bases para Programación V, que profundiza en arquitectura de aplicaciones, frameworks, patrones de diseño, pruebas y despliegue; por ello, en esta asignatura se privilegia la comprensión de los fundamentos por sobre el uso intensivo de frameworks. Asimismo, el trabajo final integrador puede dar continuidad al análisis y diseño de un sistema elaborado en Ingeniería de Software I, de modo que los estudiantes implementen un software a partir de una especificación construida previamente y recorran así el camino completo desde el análisis hasta el sistema en funcionamiento.

Articulación horizontal. Se dicta en simultáneo con Redes de Datos, que aborda los sistemas cliente-servidor y el modelo computacional de la web desde la perspectiva de los protocolos y la infraestructura, y con Bases de Datos I. Se prevé coordinar con ambas cátedras la secuencia de temas, de modo que el protocolo HTTP y la arquitectura cliente-servidor se trabajen de forma

complementaria, y que la persistencia de datos se aborde en Programación IV en un nivel introductorio, orientado a su uso desde la aplicación, para luego profundizarse en Bases de Datos I.

2.-EJES/COMPETENCIAS DE LA CARRERA A LOS QUE APORTA LA ASIGNATURA

Considerando la ubicación de la asignatura dentro del plan de estudios y del área, la misma aporta a competencias específicas del área Algoritmos y Lenguajes (CE.AyL), a competencias tecnológicas (CT) y competencias sociales, políticas y actitudinales (CSPA), organizando la formación disciplinar en torno a esos ejes, las cuales se detallan a continuación:

EJES/COMPETENCIAS	APORTE		
	ALTO	MEDIO	BAJO
CE.AyL. 2: Interpretar especificaciones de modelos para el diseño e implementación de programas mediante la correcta aplicación de principios de los paradigmas de programación.		X	
CE.AyL. 4: Evaluar lenguajes de programación para determinar su grado de adecuación a un contexto de desarrollo específico analizando las características propias del paradigma y de la herramienta.		X	
CT 1. Identificación, formulación y resolución de problemas de informática.	X		
CT 2. Concepción, diseño y desarrollo de proyectos de informática.		X	
CT 4. Utilización de técnicas y herramientas de aplicación en la informática.		X	
CT 5. Generación de desarrollos tecnológicos e innovaciones tecnológicas.		X	
CSPA 6. Fundamentos para el desempeño en equipos de trabajo.	X		
CSPA 7. Fundamentos para la comunicación efectiva.		X	
CSPA 8. Fundamentos para la acción ética y responsable.		X	
CSPA 10. Fundamentos para el aprendizaje continuo.		X	

3.- RESULTADOS DE APRENDIZAJE

Al finalizar la asignatura, de acuerdo a los contenidos teóricos y prácticos que se proponen el/la estudiante deberá alcanzar los siguiente Resultados de Aprendizaje (RA)

- RA 1: Desarrolla aplicaciones web integrales mediante la implementación de arquitecturas cliente-servidor para resolver problemas de intercambio de información siguiendo los estándares de los protocolos de comunicación actuales.

- RA 2: Crea interfaces de usuario interactivas y adaptativas utilizando lenguajes de marcado, estilos y programación asíncrona del lado del cliente, aplicando criterios de usabilidad y accesibilidad, para optimizar la experiencia de usuario según principios de diseño moderno.

- RA 3: Construye servicios web y APIs robustas mediante la programación del lado del servidor y la gestión de persistencia para garantizar la disponibilidad y seguridad de los datos bajo estándares REST, de autenticación y de seguridad, verificando su funcionamiento mediante pruebas automatizadas.

- RA 4: Optimiza el rendimiento de aplicaciones mediante la aplicación de modelos de concurrencia y algoritmos paralelos, evaluando la mejora obtenida con métricas de aceleración (speedup) y eficiencia, para maximizar la capacidad de cómputo y la respuesta del sistema en entornos multihilo.

- RA 5: Diseña sistemas distribuidos mediante el uso de mecanismos de comunicación entre nodos, WebSockets y algoritmos de coordinación para asegurar la interoperabilidad y la tolerancia a fallos en infraestructuras de red complejas.

4.-CONTENIDOS MÍNIMOS (PLAN DE ESTUDIOS)

Sistemas web. Aplicaciones cliente-servidor. Algoritmos distribuidos y paralelos. Programación distribuida y paralela. Elementos de diseño de interfaz de usuario.

5.-PROGRAMA ANALÍTICO (UNIDADES/MÓDULOS/TEMAS)

Unidad 1: Fundamentos de la Web y diseño de interfaces de usuario
Internet y World Wide Web. Modelo Cliente/Servidor en la Web. Componentes de la Web. Protocolo HTTP. Patrón de comunicación, concepto de mensajes, métodos. Estructuración de documentos HTML. Elementos básicos y semánticos. Estilos con CSS. Disposición de elementos. Frameworks CSS. Técnicas de Diseño Responsivo. Elementos de diseño de interfaz de usuario: principios de diseño visual (jerarquía, contraste, alineación y consistencia). Heurísticas de usabilidad. Patrones de interacción y de navegación. Accesibilidad web: pautas WCAG, HTML semántico y ARIA. Prototipado de baja y alta fidelidad.
Resultados de aprendizaje asociados: RA 1, RA 2.

Unidad 2: Programación del lado del cliente
JavaScript. Sintaxis y construcciones básicas. Funciones y Manejo de Eventos. Modelo de ejecución asincrónico: event loop, callbacks, promesas y async/await. Consumo de APIs desde el cliente: DOM y Fetch. Definición, procesamiento y validación de formularios y retroalimentación al usuario.
Resultados de aprendizaje asociados: RA 1, RA 2.

Unidad 3: Programación del lado del servidor
Características de la programación del lado del servidor web. Procesamiento de solicitudes HTTP. Entrega de recursos estáticos. Construcción de APIs REST. Buenas prácticas en el diseño de APIs REST. Seguridad y manejo de errores. Autenticación y Autorización. Persistencia de datos desde la aplicación: acceso a bases de datos mediante controladores (nivel introductorio). Seguridad en aplicaciones web: HTTPS, CORS y principales vulnerabilidades (OWASP Top 10: inyección, XSS, CSRF). Pruebas automatizadas de APIs.
Resultados de aprendizaje asociados: RA 1, RA 3.

Unidad 4: Concurrencia y programación paralela
Concurrencia y paralelismo: conceptos y diferencias. Modelos de concurrencia: hilos y memoria compartida, paso de mensajes, event loop y programación asincrónica, worker threads en Node.js. Sincronización en memoria compartida: condiciones de carrera, exclusión mutua, locks y semáforos. Algoritmos paralelos: divide y vencerás y fork-join. Medición del rendimiento: aceleración (speedup), eficiencia y ley de Amdahl.
Resultados de aprendizaje asociados: RA 4.

Unidad 5: Sistemas distribuidos
Fundamentos y desafíos de los sistemas distribuidos: ausencia de memoria compartida y de reloj global, fallas parciales. Comunicación entre nodos: sincrónica (REST), asincrónica (colas de mensajes) y en tiempo real (WebSockets). Tiempo y orden de eventos: relojes lógicos de Lamport. Coordinación: elección de líder (algoritmo Bully). Replicación y consistencia. Tolerancia a fallos y nociones de consenso. Particiones y disponibilidad: teorema CAP.
Resultados de aprendizaje asociados: RA 5.

6.-BIBLIOGRAFÍA

Obligatoria

- Connolly, R., & Hoar, R. (2022). *Fundamentals of web development* (3.ª ed.). Pearson.
- Flanagan, D. (2020). *JavaScript: The definitive guide* (7.ª ed.). O'Reilly Media.
- Haverbeke, M. (2024). *Eloquent JavaScript* (4.ª ed.). No Starch Press.
- Brown, E. (2019). *Web development with Node and Express: Leveraging the JavaScript stack*. O'Reilly Media.
- Coulouris, G., Dollimore, J., & Kindberg, T. (2012). *Distributed systems: Concepts and design* (5.ª ed.). Pearson.
- Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.
- Hunter, T., & English, B. (2021). *Multithreaded JavaScript: Concurrency beyond the event loop*. O'Reilly Media.
- van Steen, M., & Tanenbaum, A. S. (2023). *Distributed systems* (4.ª ed.). distributed-systems.net. Disponible en acceso abierto.
- Tidwell, J., Brewer, C., & Valencia, A. (2019). *Designing interfaces: Patterns for effective interaction design* (3.ª ed.). O'Reilly Media.

Complementaria

- Meyer, E. A. (2017). *CSS: The definitive guide* (4.ª ed.). O'Reilly Media.
- Marcotte, E. (2014). *Responsive web design* (2.ª ed.). A Book Apart.
- Masse, M. (2011). *REST API design rulebook*. O'Reilly Media.
- Grinberg, M. (2018). *Flask web development* (2.ª ed.). O'Reilly Media.
- Krug, S. (2014). *Don't make me think, revisited: A common sense approach to web usability* (3.ª ed.). New Riders.
- W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/
- Mozilla. (s.f.). *MDN Web Docs* [Documentación en español]. https://developer.mozilla.org/es/

7.- METODOLOGÍA DE ENSEÑANZA

La propuesta pedagógica se basa en un enfoque Basado en Competencias y Centrado en el Estudiante, promoviendo su participación activa y el desarrollo de competencias vinculadas al ejercicio profesional siempre en coherencia con los alcances del título y las actividades profesionales reservadas.

Los contenidos teóricos y prácticos se abordarán de manera articulada mediante estrategias didácticas orientadas a la resolución de problemas, actividades aplicadas y prácticas de laboratorio, haciendo uso de herramientas y tecnologías actualmente demandadas por la industria del desarrollo de software y destacando las experiencias profesionales de los profesores y estudiantes.

Como soporte tecnológico se utilizarán software de código abierto, los equipos de las salas de máquinas de nuestra Facultad de Ciencias de la Administración y el Campus Virtual UNER, que funcionará como espacio para el dictado de clases, la publicación de materiales, la realización de autoevaluaciones, la entrega de proyectos y la comunicación entre docentes y estudiantes.

Estrategias de enseñanza. Se desarrollarán clases teórico-prácticas con demostraciones de código en vivo y actividades prácticas breves en clase (HTML, CSS, JavaScript, Node.js y Express, y luego ejemplos de concurrencia y de algoritmos distribuidos), pensadas para que las dudas surjan y se resuelvan en el momento. En paralelo, los estudiantes desarrollan en equipos un trabajo final integrador incremental, que avanza semana a semana acompañando la secuencia de las unidades: comienza como una aplicación web cliente-servidor y evoluciona hacia una versión concurrente y distribuida. El uso de control de versiones (Git) y la revisión de código entre pares se incorporan como prácticas habituales de trabajo.

Distribución de la carga horaria. De las 128 horas totales, aproximadamente 48 horas (37,5 %) se destinan a actividades teóricas y 80 horas (62,5 %) a formación práctica (laboratorio, resolución de problemas y desarrollo del proyecto integrador), lo que supera el mínimo del 50 % de formación práctica establecido por el plan de estudios.

Uso de herramientas de inteligencia artificial generativa. Los asistentes de programación basados en inteligencia artificial forman parte del ejercicio profesional actual, por lo que su uso se incorpora de manera explícita y guiada. Se habilitan en actividades determinadas como apoyo para explorar alternativas, depurar y documentar código, y se exige que los estudiantes declaren su uso y sean capaces de explicar, justificar y modificar todo el código que entregan. Las instancias de evaluación individual (parcial y defensa del trabajo final integrador) verifican esa comprensión. Se promueve, además, una mirada crítica sobre la calidad, la seguridad y las implicancias éticas del código generado.

8. - METODOLOGÍA DE EVALUACIÓN

La evaluación se concibe como una instancia integral del proceso de enseñanza-aprendizaje, no limitada a la acreditación, sino orientada al aprendizaje continuo y a la mejora progresiva.

A lo largo de la asignatura se evaluará permanentemente el proceso de enseñanza-aprendizaje. En este sentido, la evaluación no se considera limitada sino que se entiende como un proceso integral donde la misma es formativa y de seguimiento, tanto en actividades teóricas como prácticas, enfocada en el desarrollo de competencias profesionales propias del área de la programación.

Se considerarán como criterios mínimos de evaluación los siguientes aspectos:

- Comprensión conceptual y aplicación del conocimiento: se valorará el dominio de los conceptos fundamentales vinculados al desarrollo web tanto del lado del cliente como del servidor, los sistemas distribuidos y la programación concurrente, así como la capacidad para aplicarlos en la resolución de problemas concretos y el uso adecuado del lenguaje técnico.
- Desarrollo de habilidades prácticas: se evaluará la correcta implementación de aplicaciones y servicios, el uso apropiado de herramientas y entornos de desarrollo, la resolución eficiente de consignas prácticas y la transferencia de los conceptos teóricos a soluciones funcionales y escalables.
- Trabajo colaborativo y participación activa: se considerará la participación en clase, el trabajo en equipo, la capacidad para integrar aportes en proyectos grupales, la responsabilidad en el cumplimiento de tareas y el respeto por buenas prácticas de desarrollo de software.
- Comunicación técnica y académica: se valorará la claridad y coherencia en la documentación, presentaciones y entregas, el uso adecuado del lenguaje técnico, así como el cumplimiento de plazos y consignas establecidas.

Instrumentos de evaluación. Se utilizarán rúbricas para los trabajos prácticos y el proyecto integrador, conocidas por los estudiantes desde el inicio de la cursada; autoevaluaciones en el Campus Virtual; un parcial individual de carácter teórico-práctico; y la defensa oral del trabajo final integrador.

9. -RÉGIMEN DE EVALUACIÓN Y ACREDITACIÓN

Acorde a lo establecido en el Reglamento Académico vigente Resolución "C.D." 509/16, y modificatorias (t.o. por la Resolución de Decano Nº 310/21), a fines de la regularidad se considerarán:

- la entrega en tiempo y forma y aprobación de trabajos prácticos, actividades de laboratorio y autoevaluaciones de seguimiento.
- la aprobación de un (1) parcial o su correspondiente recuperatorio, de carácter individual y teórico-práctico. Se opta por una única instancia de parcial porque el proyecto integrador constituye el principal instrumento de integración de los contenidos.

Para la acreditación de la asignatura se establece un régimen de promoción directa. En este marco, el/la estudiante deberá cumplir con los requisitos de regularidad mencionados previamente y, además, presentar y aprobar un trabajo final integrador. En caso de no aprobar dicho trabajo, el/la estudiante deberá rendir el examen final en los turnos establecidos por el calendario académico, a fin de alcanzar la acreditación de la asignatura. El trabajo final integrador se desarrolla en equipos de hasta cinco integrantes, consiste en la evolución del proyecto realizado durante el cursado e incorpora componentes concurrentes y distribuidos. Se aprueba mediante una defensa oral individual, en la que cada integrante debe dar cuenta del diseño y del código, con una calificación igual o superior a 6 (seis). Los estudiantes regulares que no alcancen la promoción rendirán un examen final que consiste en la presentación y defensa del trabajo final integrador y en preguntas conceptuales sobre los contenidos del programa.

Aquellos/as estudiantes que no cumplan con las condiciones de regularidad quedarán en condición de libre, pudiendo presentarse directamente a mesa de examen final en los turnos correspondientes. En ese caso, deberán aprobar previamente una instancia escrita de resolución de problemas prácticos y, luego, un examen oral sobre la totalidad de los contenidos del programa.

En todas las instancias de evaluación se aplicará la escala de calificaciones vigente en la universidad.